

# Software Evolution — Phase 2 Report

Search Result Counter Enhancement — UniTime System

---

|                |                                                                                        |
|----------------|----------------------------------------------------------------------------------------|
| Course         | Software Evolution                                                                     |
| Supervisor     | Dr. Lamia Abo Zaid                                                                     |
| Project        | UniTime University Timetabling System (v4.9)                                           |
| Feature branch | SearchResultCounter → main                                                             |
| Pull Request   | #5 — <a href="https://github.com/ahmedramy9/unitime">github.com/ahmedramy9/unitime</a> |
| Date           | May 9, 2026                                                                            |

## Team IDs:

20236002

20236005

20236009

20236056

## 1. What We Did

---

UniTime is an open-source university timetabling system. For this phase we picked a small but practical usability gap in the Class Search page: when a user ran a search, the page showed the results table but gave no indication of how many classes matched. We added a count message that appears just above the table after every successful search — for example, *Found 12 classes* — and disappears when there are no results.

On the implementation side, three files were changed, adding eight lines in total. No existing logic was removed and no database changes were made. We then reverse-engineered UML diagrams of the affected subsystem, carried out a formal impact analysis following the methodology from the course lecture, and pushed everything to a Git branch that was merged via a pull request.

## 2. Files Changed

---

| File                                | Change                                                                                                                                   |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ClassSearchAction.java</code> | Added <code>request.setAttribute("resultCount", classes.size())</code> in both the automatic and manual search paths (lines 200 and 306) |
| <code>CourseMessages.java</code>    | Added a new localization method: <code>@DefaultMessage("Found {0} classes") String foundClasses()</code>                                 |
| <code>classSearch.jsp</code>        | Added a conditional block that reads the request attribute and renders the count banner above the table                                  |

```
// classSearch.jsp - new block
<s:if test="showTable == true && #request.resultCount != null">
  <div style="padding:8px; background:#f0f0f0;">
    <strong>
      <loc:message name="foundClasses">
        <s:param value="%{#request.resultCount}"/>
      </loc:message>
    </strong>
  </div>
</s:if>
```

### 3. UML Diagrams

---

We reverse-engineered three diagrams from the source code. The diagrams were built by reading the Java source, Hibernate mappings, configuration files, and JSP views, then modelling the structure in PlantUML.

#### 3.1 System Component Diagram

Shows the overall architecture of UniTime — the servlet container, filter chain, MVC layers, Spring context, subsystems, solver, and database. The Course Timetabling subsystem (where our change lives) is highlighted.

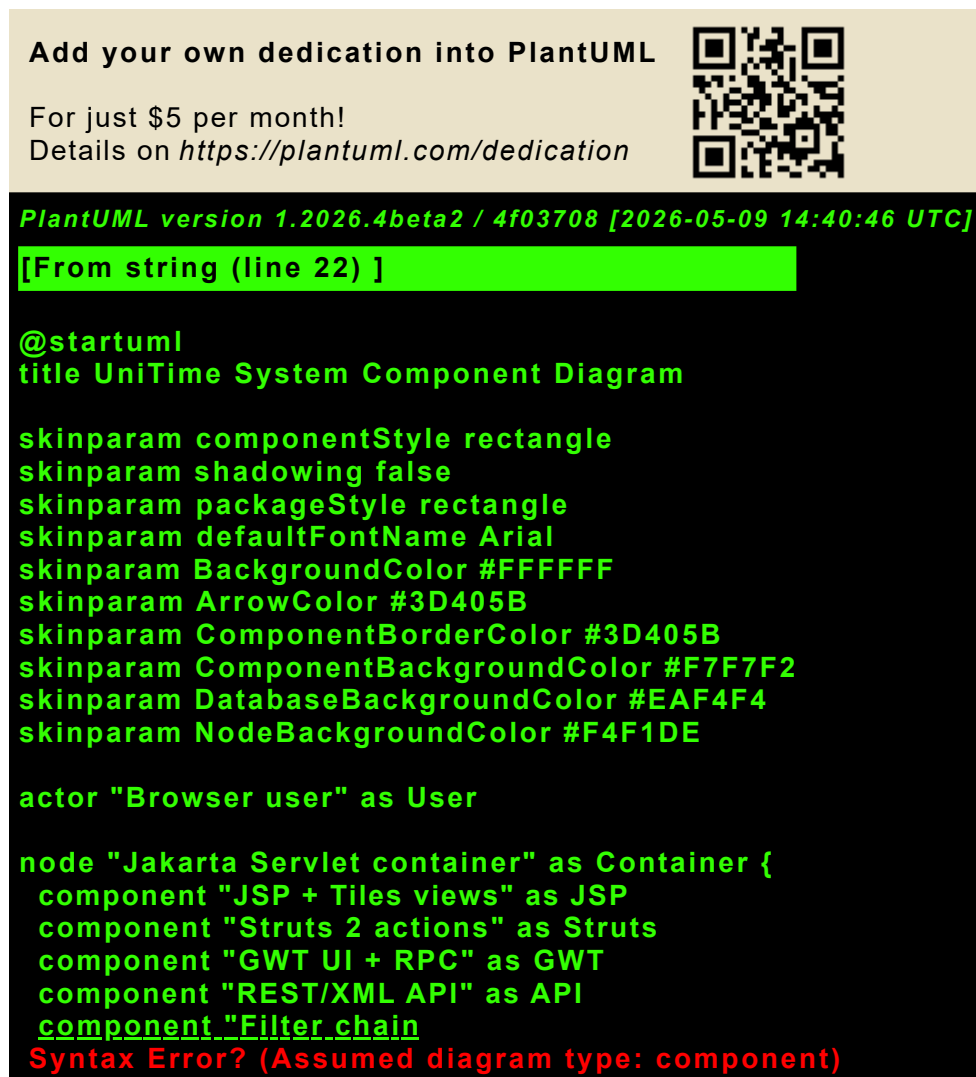


Figure 1 — UniTime system component diagram (reverse-engineered)

#### 3.2 Class Search Subsystem — Class Diagram

Shows the classes involved in handling a class search request. Elements marked «new» were added or changed by the SearchResultCounter feature.

Course Timetabling — Class Search subsystem  
(«new» marks SearchResultCounter additions)

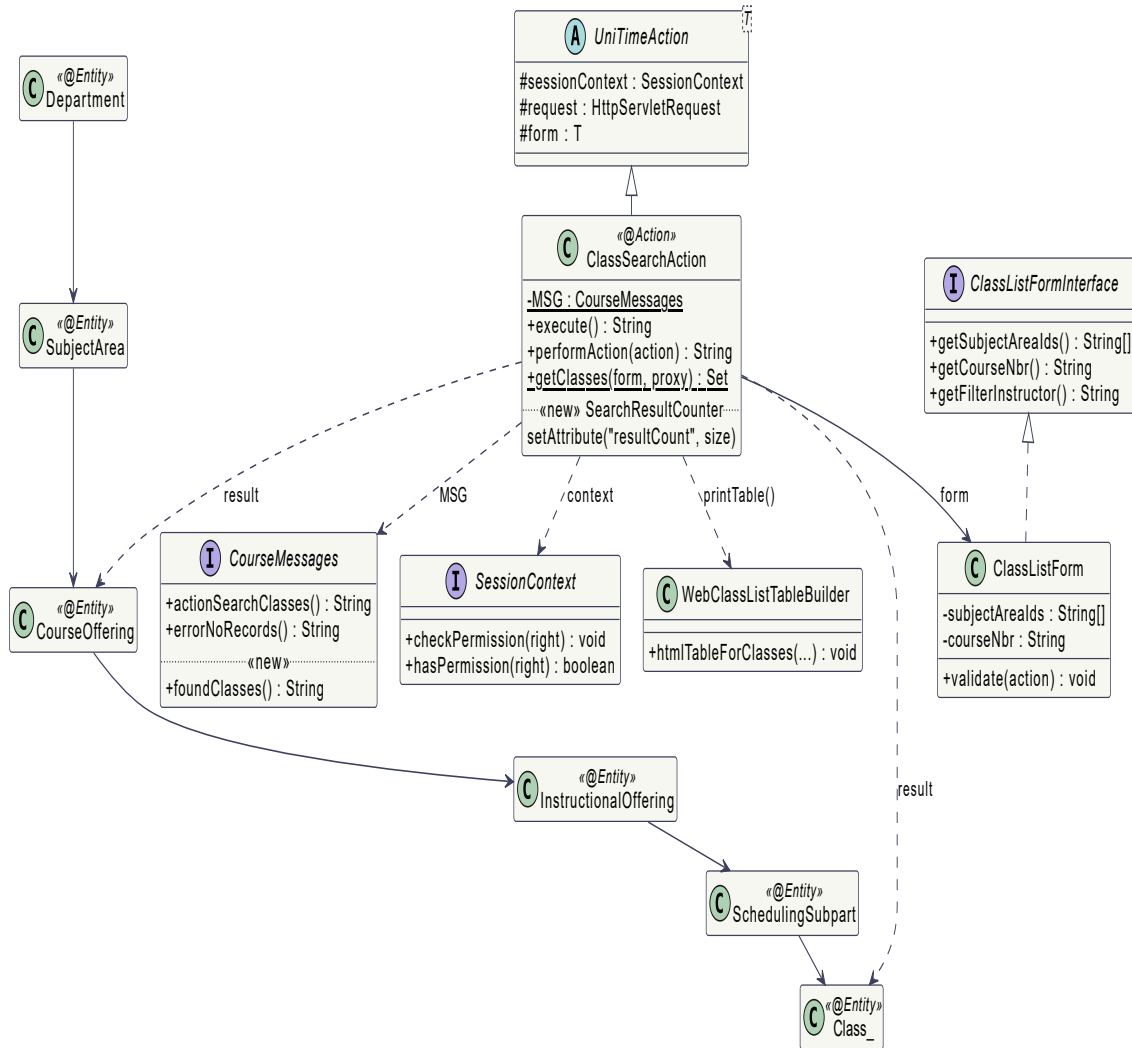


Figure 2 — Course Timetabling / Class Search class diagram

### 3.3 Class Search Request Flow — Sequence Diagram

Traces the full lifecycle of a search request, from form submission to page render. The two steps marked «new» show where the count is stored and where the JSP displays it.

[From string (line 107) ]

```
@startuml
title Class Search request sequence\n(<<new>> steps added by SearchResultCounter)

skinparam sequenceMessageAlign center
skinparam shadowing false
...
... ( skipping 79 lines )
...
    Builder -> DB : load room / instructor / assignment data
    DB --> Builder : assignment details
    Builder --> JSP : HTML table rows
    JSP -> User : render full class results table
end

User -> JSP : POST classSearch.action (doit="Export PDF")
JSP -> Action : performAction("exportPdf")
note right of Action
    getClasses() called again.
    resultCount still set in request scope
    (attribute present but JSP not rendered for export).
end note
Action -> DB : HQL query (same as search)
DB --> Action : classes
Action -> Builder : pdfTableForClasses(...)
Builder --> User : PDF file download

note bottom
    Syntax Error?
```

Figure 3 — Class Search sequence diagram

### 3.4 What Changed in the UML

| Element                        | Before                         | After                                                         |
|--------------------------------|--------------------------------|---------------------------------------------------------------|
| <code>ClassSearchAction</code> | Fetches classes, returns view  | Also stores result count in request scope                     |
| <code>CourseMessages</code>    | No <code>foundClasses()</code> | New method: <code>foundClasses()</code> → "Found {0} classes" |
| <code>classSearch.jsp</code>   | Table only                     | Count banner + table                                          |

## 4. Impact Analysis

---

The analysis follows the formal process from the course lecture (IEEE/EIA 1219 / Tripathy & Naik Ch. 6), working from a Starting Impact Set through to metrics.

### 4.1 Starting Impact Set (SIS)

The CR asks for a count on the search results page. The two most obvious components to look at first are the action that runs the search and the JSP that renders the page.

```
SIS = { ClassSearchAction.java, classSearch.jsp }    |SIS| = 2
```

### 4.2 Tracing Dependencies → Candidate Impact Set (CIS)

Following the call chain from the SIS, two more files came up as candidates:

- `CourseMessages.java` — the JSP already uses this for all user-facing text, so a new message key is clearly needed.
- `ClassListForm.java` — holds the classes collection; included as a precaution in case the count needed to be stored on the form.

```
CIS = { ClassSearchAction.java, classSearch.jsp, CourseMessages.java,
ClassListForm.java }    |CIS| = 4
```

### 4.3 Actual Impact Set (AIS)

After implementing the change, `ClassListForm.java` turned out not to need modification — the count was available directly from the existing collection. Everything else matched the prediction.

```
AIS = { ClassSearchAction.java, classSearch.jsp, CourseMessages.java }
|AIS| = 3
```

### 4.4 Discovered and False Positive Sets

```
DIS = { }    |DIS| = 0 — no surprises during implementation
```

```
FPIS = (CIS ∪ DIS) - AIS = { ClassListForm.java }    |FPIS| = 1
```

### 4.5 Metrics

| Metric | Formula                  | Value      | What it means                                    |
|--------|--------------------------|------------|--------------------------------------------------|
| Recall | $ CIS \cap AIS  /  AIS $ | <b>1.0</b> | Every modified file was predicted. DIS is empty. |

|               |                          |               |                                                         |
|---------------|--------------------------|---------------|---------------------------------------------------------|
| Precision     | $ CIS \cap AIS  /  CIS $ | <b>0.75</b>   | One false positive (ClassListForm) reduced precision.   |
| Inclusiveness | $AIS \subseteq CIS$      | <b>1</b>      | No actual change was missed.                            |
| Amplification | $ IISO  /  DISO $        | <b>0</b>      | No ripple effect — change is fully contained.           |
| ChangeRate    | $ CIS  /  System $       | <b>0.0016</b> | Only 0.16% of system objects considered (highly sharp). |
| S-Ratio       | $ AIS  /  CIS $          | <b>0.75</b>   | Good fit between prediction and reality.                |

## 4.6 Ripple Effect

The change introduced no ripple effect. The new `resultCount` attribute is write-only and does not feed back into any search logic. The `foundClasses()` method is brand new, so no existing code references it. No new classes, queries, or relationships were introduced, and no existing workflows were touched. The risk of error propagation is effectively zero.

## 4.7 Change Propagation

| Step | Component                           | What happened                                          |
|------|-------------------------------------|--------------------------------------------------------|
| 1    | <code>ClassSearchAction.java</code> | Stores the result count in the HTTP request scope      |
| 2    | <code>CourseMessages.java</code>    | Defines the localized display message                  |
| 3    | <code>classSearch.jsp</code>        | Reads the count and renders the banner above the table |
| 4    | —                                   | No further propagation                                 |

## 4.8 Regression Testing Scope

Only the three modified components need to be tested. Student Scheduling, Exam Timetabling, Event Management, the REST API, and the solver engine are all outside the AIS and unaffected.

| Component                           | Test focus                                                                                           |
|-------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>ClassSearchAction.java</code> | Both search paths set <code>resultCount</code> correctly after a successful query                    |
| <code>classSearch.jsp</code>        | Count banner appears when results exist; hidden when search returns nothing; export paths unaffected |
| <code>CourseMessages.java</code>    | Parameterized message formats correctly (e.g. "Found 5 classes")                                     |

## 5. Verification

| Check                                                                                             | Result |
|---------------------------------------------------------------------------------------------------|--------|
| <code>request.setAttribute("resultCount", ...)</code> present in automatic search path (line 200) | ✓      |
| <code>request.setAttribute("resultCount", ...)</code> present in manual search path (line 306)    | ✓      |
| <code>foundClasses()</code> added to <code>CourseMessages.java</code> (line 1948)                 | ✓      |
| Conditional <code>&lt;s:if&gt;</code> block in <code>classSearch.jsp</code> (lines 341–345)       | ✓      |
| Search with results → banner shows correct count                                                  | ✓      |
| Search with no results → no banner shown                                                          | ✓      |
| Export PDF / CSV → downloads work normally, no banner                                             | ✓      |
| Other pages (Exams, Events, Scheduling) → no change in behaviour                                  | ✓      |

## 6. Git and Pull Request

All work was committed to the `SearchResultCounter` branch and merged to `main` via Pull Request #5. The PR description included a summary of changes, a before/after table, and a test checklist.

| Commit               | Description                                         |
|----------------------|-----------------------------------------------------|
| <code>92c1bfa</code> | Add SearchResultCounter feature                     |
| <code>f533027</code> | Add UML diagrams and impact analysis                |
| <code>17781b0</code> | Rewrite impact analysis to match formal methodology |
| <code>5b671bc</code> | Clean up Documentation/UML folder                   |